



Análisis de Retención y Comportamiento de Clientes (Churn Check)



Estudio detallado sobre la lealtad y abandono de clientes

- Fecha limite: 27/01/2026
- Importante: Estos entregables son documentos vivos y editables.
- Mantén los documentos actualizados y visibles para las empresas durante todo el proyecto. No olvides la fecha límite de entrega.

Tarea 1: Documentación del Proyecto

- Escribe información del proyecto y enlaces en formato Markdown

Tarea 2: Video Demo del Proyecto

Sube el enlace de YouTube para la demostración del proyecto

Tarea 3: Herramientas del Equipo

- Selecciona las herramientas y tecnologías que el equipo utilizó en el proyecto
- Consejo: Selecciona hasta 10 herramientas que el equipo ha utilizado en el proyecto



- Sector de negocio

Servicios y Suscripciones (Telecomunicaciones, Fintech, Streaming, E-commerce)

Empresas que dependen de clientes recurrentes y desean reducir cancelaciones o desistencias.

- Descripción del proyecto

El desafío de **ChurnInsight** es crear una solución que **prediga si un cliente es propenso a cancelar un servicio (churn)**.

El objetivo es que el equipo de **Data Science** desarrolle un modelo predictivo y que el equipo de **Back-end** construya una **API** para disponibilizar esa predicción a otros sistemas, permitiendo que el negocio actúe antes de que el cliente decida irse.

Ejemplo: una fintech quiere saber, basándose en los hábitos de uso e historial de pago, qué clientes tienen alta probabilidad de deserción. Con esta información, el equipo de marketing puede ofrecer servicios personalizados y el equipo de soporte puede actuar preventivamente.

- Necesidad del cliente (explicación no técnica)

Toda empresa que vende por suscripción o contrato recurrente sufre con cancelaciones. Mantener clientes fieles es más barato que conquistar nuevos.

El cliente (empresa) quiere **predecir con anticipación** quién está a punto de cancelar, para poder actuar y retener a esas personas.

La solución esperada debe ayudar a:

identificar clientes con riesgo de churn (cancelación);

priorizar acciones de retención (ofertas, contactos, bonos);

medir el impacto de estas acciones a lo largo del tiempo.



- Validación de mercado

La predicción de *churn* es una de las aplicaciones más comunes y valiosas de la ciencia de datos en negocios modernos.

Empresas de telecomunicaciones, bancos digitales, gimnasios, plataformas de streaming y proveedores de software utilizan modelos de churn para:

reducir pérdidas financieras; entender patrones de comportamiento de clientes; aumentar el tiempo promedio de relación (lifetime value).

Incluso modelos simples ya aportan valor, pues ayudan a las empresas a dirigir esfuerzos donde hay mayor riesgo de pérdida.

- Expectativa para este hackathon

Público: estudiantes principiantes en tecnología, sin experiencia profesional en el área, pero que ya estudiaron Back-end con Java (APIs REST, persistencia, pruebas) y Data Science (Python, Pandas, scikit-learn, ML supervisado).

Objetivo: construir, en grupo, un MVP (producto mínimo viable) capaz de predecir si un cliente va a cancelar y disponibilizar esa predicción a través de una API funcional.

Alcance ideal: clasificación binaria ("va a cancelar" / "va a continuar") con base en un dataset pequeño y limpio.



Entregables deseados

- Notebook (Jupyter/Colab) del equipo de Data Science, que contenga:

Exploración y limpieza de los datos (EDA);

Ingeniería de features (ej.: tiempo de uso, frecuencia de login, historial de pago);

Entrenamiento de modelo supervisado (ej.: Logistic Regression, Random Forest);

Métricas de desempeño (Accuracy, Precision, Recall, F1-score);

Serialización del modelo (joblib/pickle).

- Aplicación Back-End (API REST) del equipo de Java:

Endpoint que recibe información de un cliente y devuelve la predicción del modelo (Ej.: "Va a cancelar" / "Va a continuar");

Integración con el modelo de DS (directa o vía microservicio Python);

Logs y manejo de errores.

- Documentación mínima (README):

Cómo ejecutar el modelo y la API;

Ejemplos de petición y respuesta (JSON);

Dependencias y versiones de las herramientas.

- Demostración funcional (Presentación corta):

Mostrar la API en acción (a través de Postman, cURL o interfaz simple);

Explicar cómo el modelo llega a la predicción.



Entregables deseados

- Notebook (Jupyter/Colab) del equipo de Data Science, que contenga:

Exploración y limpieza de los datos (EDA);

Ingeniería de features (ej.: tiempo de uso, frecuencia de login, historial de pago);

Entrenamiento de modelo supervisado (ej.: Logistic Regression, Random Forest);

Métricas de desempeño (Accuracy, Precision, Recall, F1-score);

Serialización del modelo (joblib/pickle).

- Aplicación Back-End (API REST) del equipo de Java:

Endpoint que recibe información de un cliente y devuelve la predicción del modelo (Ej.: "Va a cancelar" / "Va a continuar");

Integración con el modelo de DS (directa o vía microservicio Python);

Logs y manejo de errores.

- Documentación mínima (README):

Cómo ejecutar el modelo y la API;

Ejemplos de petición y respuesta (JSON);

Dependencias y versiones de las herramientas.

- Demostración funcional (Presentación corta):

Mostrar la API en acción (a través de Postman, cURL o interfaz simple);

Explicar cómo el modelo llega a la predicción.



Funcionalidades exigidas (MVP)

El servicio debe exponer un endpoint que devuelve una predicción sobre el cliente y la probabilidad asociada a esa predicción. Ejemplo: POST /predict: recibe JSON con datos del cliente y devuelve:

```
{ "prevision": "Va a cancelar", "probabilidad": 0.76 }
```

Carga de modelo predictivo: el back-end debe ser capaz de acceder al modelo de churn (localmente o vía servicio DS).

Validación de entrada: verificar si todos los campos obligatorios están llenos.

Respuesta estructurada: incluir predicción y probabilidad numérica.

Ejemplos de uso: 3 peticiones de prueba (clientes con y sin cancelación).

Documentación simple: un README explicando cómo ejecutar el proyecto y reproducir las pruebas.

Funcionalidades opcionales

Endpoint GET /stats: devuelve estadísticas básicas, como: { "total_evaluados": 500, "tasa_churn": 0.23 }

Persistencia de predicciones: almacenar clientes y resultados en base de datos (H2 o PostgreSQL).

Dashboard simple (Streamlit o HTML): visualiza clientes con mayor riesgo.

Explicabilidad básica: incluir en el retorno las 3 variables más relevantes para el resultado (ej.: "tiempo de contrato", "retrasos en pagos", "uso de la app").

Batch Prediction: endpoint que acepta lista de clientes (archivo CSV).

Contenerización: ejecutar el sistema completo con Docker/Docker Compose.

Pruebas automatizadas: unitarias y de integración simples (JUnit, pytest).



Orientaciones técnicas para estudiantes

Controlar el volumen de datos y el uso de OCI, teniendo en cuenta la cantidad de memoria que OCI soporta, cuidando los datos utilizados para no extrapolar el Free-Tier de OCI.

Equipo de Data Science:

Arme o elija un dataset propio con información de clientes (ejemplo: tiempo de contrato, retrasos en pago, uso del servicio, tipo de plan, etc.).

Utilizar Python, Pandas y scikit-learn para análisis y modelado.

Elegir modelo simple de clasificación (LogisticRegression, RandomForest);

Crear features intuitivas (ej.: tiempo de cliente, número de compras recientes, promedio de gastos);

Guardar modelo y pipeline (joblib.dump) y garantizar que pueda ser cargado fuera del notebook.

Equipo de Back-end:

Construir una API REST (Java + Spring Boot); Recibir JSON con datos de cliente y devolver la predicción;

Conectarse al modelo de DS:

vía microservicio Python (FastAPI/Flask), o cargando modelo exportado en formato ONNX (opción más avanzada); Validar entradas y devolver errores claros cuando falte información.

Contrato de integración (JSON)

Recomendamos definir el contrato de integración justo al inicio del hackathon. Sigue un ejemplo:

Entrada:{"tiempo_contrato_meses": 12,"retrasos_pago": 2,"uso_mensual": 14.5,"plan": "Premium"}

Salida:{"prevision": "Va a cancelar","probabilidad": 0.81}